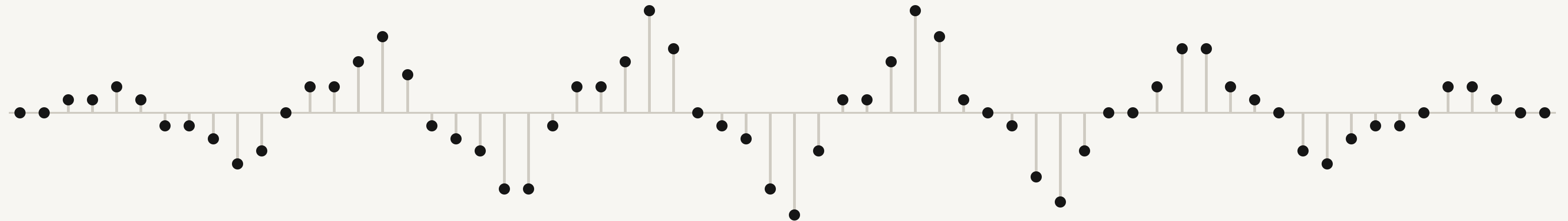


# Demystifying Game Audio Codecs

What they solve, how to choose, and what's next

**Collin Schupman, Senior Engineer**

GameSoundCon · Wednesday, October 21, 2026



# About me

I've worked on game audio in commercial engines and proprietary ones.

|                          |                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------|
| <b>Riot Games</b>        | Unreal Audio, Wwise<br>Unannounced R&D                                                 |
| <b>High Moon Studios</b> | Proprietary engine tech<br>Call of Duty: Modern Warfare 4, Black Ops 6 and Black Ops 7 |
| <b>Gas Powered Games</b> | Unity and Wwise<br>Unpublished title                                                   |

# What a codec setting affects

Most projects pick a codec once, from a dropdown. That one setting decides:

|                            |                                             |
|----------------------------|---------------------------------------------|
| <b>File size</b>           | Install footprint and patch size            |
| <b>Streaming</b>           | Disk and I/O bandwidth under load           |
| <b>Memory</b>              | RAM held by loaded sounds                   |
| <b>CPU</b>                 | Decode cost for every voice that is playing |
| <b>Platform compliance</b> | Format requirements and certification       |
| <b>Sound quality</b>       | What players actually hear                  |

# Outline

Four sections. No code, and not too deep on the technical side.

|    |                           |                                                                          |
|----|---------------------------|--------------------------------------------------------------------------|
| 01 | <b>The basics</b>         | What audio data is, and the problems codecs are meant to solve           |
| 02 | <b>The past</b>           | Codecs in games since sample playback took over, and what persists today |
| 03 | <b>The modern toolbox</b> | What teams can use now, and strategies for adopting it                   |
| 04 | <b>The frontier</b>       | What is on the horizon, and what to keep an eye on                       |

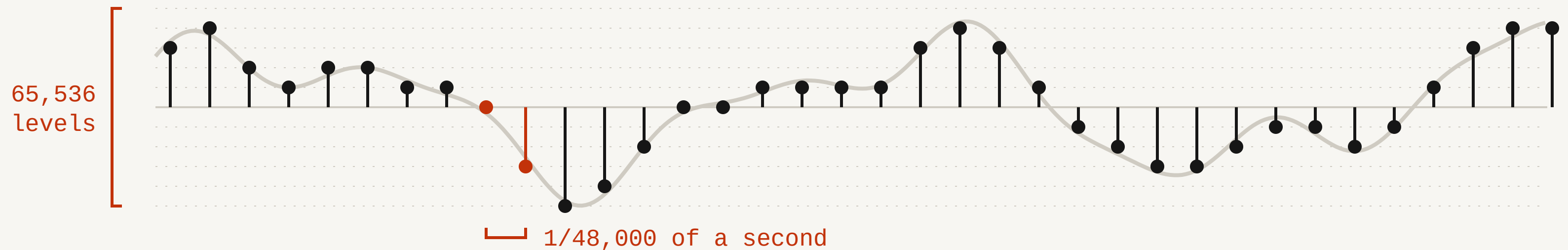
01

# The basics

Raw PCM, how codecs work and the problems they solve

# What is digital audio?

Pulse-code modulation (PCM). The amplitude of the analog signal is sampled at uniform intervals, and each sample is quantized to the nearest value within a range of digital steps.



## Sample rate

How often the wave is measured:  
48,000 times a second in most games.

## Bit depth

How precisely each measurement is  
stored. 16 bits gives 65,536 levels.

## Channels

How many waveforms play at once.  
Stereo is 2, and 5.1 is 6.

# The cost of PCM: memory

48,000

×

16

×

2

=

1,536 kbps

samples per second

bits per sample

channels

uncompressed stereo, or 192 KB every second

| Content            | Size as PCM |
|--------------------|-------------|
| One minute, stereo | 11.5 MB     |
| One hour, stereo   | 691 MB      |
| One hour, 5.1      | 2.07 GB     |

For scale: all the sound RAM a PlayStation 2 game had



That RAM would hold about 11 seconds of this audio. Then multiply by every variation, language and platform.

# The cost of PCM: streaming

Streaming means two things in games: reading audio from disk as it plays, and sending it over a network. Raw PCM makes both harder.

From disk

**192 KB/s**

per stereo stream, read constantly

- Competes for I/O with textures and level loads
- Hard drives and optical discs pay for every seek
- Each stream holds a RAM buffer: 2 s is 384 KB
- Nothing shrinks on disk: Titanfall shipped 35 GB

Over the network

**768 kbps**

per mono voice, sent constantly

- Upstream bandwidth is shared with game traffic
- A 20 ms frame (1,920 bytes) overflows one packet
- No built-in recovery: a lost packet is a gap
- Fixed bitrate: it cannot adapt to a bad connection

From disk, PCM buys you zero decode CPU. Over the internet, that is rarely worth the bandwidth.

# A tradeoff in the wild: Titanfall

48 GB

35 GB

≈ 73%

PC install size, 2014

of it was uncompressed audio

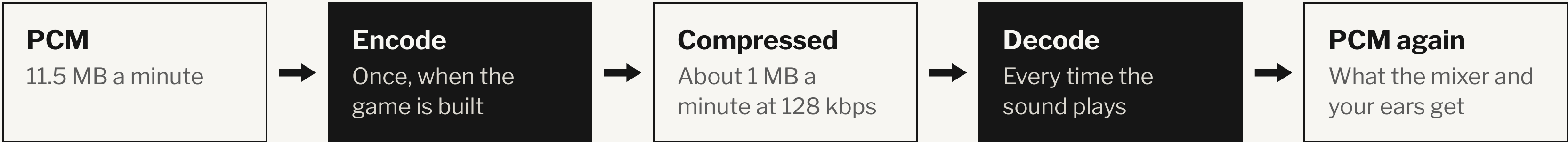
of the install

|            |                                                                   |
|------------|-------------------------------------------------------------------|
| The choice | Ship the PC audio uncompressed                                    |
| The reason | Decoding costs CPU, and the minimum-spec PC had two cores         |
| The trade  | Disk space and download size, spent to keep CPU free for the game |

“On a two-core [machine] with where our min spec is, we couldn't dedicate those resources to audio.”  
Richard Baker, lead engineer at Respawn, to Eurogamer. Via The Escapist, March 2014.

# Codec basics: types of codec

A codec is a coder and a decoder. It makes audio smaller to store or send, then turns it back into PCM to play.



| Type              | What it does                             | Compression ratio        | Decode cost | Examples          |
|-------------------|------------------------------------------|--------------------------|-------------|-------------------|
| Lossless          | Every sample comes back exactly          | About 2:1                | Low         | FLAC              |
| Lossy: predictive | Predicts, then stores a rough correction | 4:1, fixed               | Lowest      | ADPCM             |
| Lossy: perceptual | Drops what the ear won't notice          | 10:1 or more, adjustable | Highest     | Vorbis, Opus, AAC |

Compression ratio: the original size against the compressed size. 4:1 means a quarter of the size.

# Codec basics: the levers

These are the settings you can pull, and what each one changes.

|                              |                                                                           |
|------------------------------|---------------------------------------------------------------------------|
| Codec                        | Everything at once: size, CPU cost, quality, and where it can run         |
| Bitrate or quality           | Size against sound quality. Lower is smaller, until you start to hear it  |
| Constant or variable bitrate | Variable spends more where the sound is busy: better quality for the size |
| Sample rate                  | Size and CPU against high-frequency detail                                |
| Channels                     | Mono is half the size and half the decode cost of stereo                  |
| In memory or streamed        | RAM against disk I/O and start-up delay                                   |

Every codec pays for its savings in CPU, latency or quality.

# Codec basics: other gotchas

Size and CPU are not the whole story. Four more things can rule a codec out.

|                              |                                                                                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Looping</b>               | Some codecs add a sliver of silence at the start and end of a file. A seamless loop clicks or gaps unless your tool trims it. MP3 is the classic case. |
| <b>Licensing</b>             | Some codecs are covered by patents. Vorbis and Opus are royalty-free. AAC is licensed, and MP3 was until 2017.                                         |
| <b>Proprietary</b>           | Some codecs belong to one company: XMA on Xbox, ATRAC9 on PlayStation, Bink Audio in Unreal. Each platform needs its own encode.                       |
| <b>Hardware acceleration</b> | Some platforms decode certain codecs on dedicated hardware, which frees CPU for more voices. XMA on Xbox is the usual example.                         |

02

# The past

A short history of game audio codecs

# A short history

---

|                                                          |                                    |                                                   |                                         |                                         |                                            |                                               |
|----------------------------------------------------------|------------------------------------|---------------------------------------------------|-----------------------------------------|-----------------------------------------|--------------------------------------------|-----------------------------------------------|
| 1973                                                     | 1993                               | 1997                                              | 2002                                    | 2005                                    | 2011                                       | 2012                                          |
| <b>ADPCM</b>                                             | <b>MP3</b>                         | <b>AAC</b>                                        | <b>Vorbis</b>                           | <b>XMA</b>                              | <b>ATRAC9</b>                              | <b>Opus</b>                                   |
| Predictive coding from Bell Labs. Still in every engine. | Perceptual coding goes mainstream. | MP3's successor, later decoded in phone hardware. | Royalty-free. Becomes the game default. | Xbox 360's format, decoded in hardware. | Sony's low-latency codec, from PS Vita on. | IETF standard. Speech and music in one codec. |

Predictive coding first, then perceptual coding, then platform-native codecs, then one flexible open codec.

# Sample-based audio

Early consoles synthesized sound on a chip. Once games played back recordings, the sample data had to fit in very little memory.

| Year | Hardware       | Audio memory                    | Compression                           |
|------|----------------|---------------------------------|---------------------------------------|
| 1990 | Super Nintendo | 64 KB audio RAM, 8 voices       | BRR, an ADPCM variant, about 3.5:1    |
| 1994 | PlayStation    | 512 KB sound RAM, 24 voices     | 4-bit ADPCM; XA-ADPCM streams from CD |
| 1996 | Nintendo 64    | Samples share the cartridge ROM | ADPCM                                 |
| 2000 | PlayStation 2  | 2 MB sound RAM, 48 voices       | 4-bit ADPCM                           |
| 2005 | Xbox 360       | Shared system RAM               | XMA, decoded in hardware              |

ADPCM won that era because the hardware could decode it for almost nothing.

# ADPCM and MP3

Why it survives

## ADPCM

- Fixed 4:1 against 16-bit PCM
- Near-zero decode CPU
- Sample-accurate seeks and loops
- Still right for short, busy SFX

Why it faded

## MP3

- Encoder delay and padding add silence
- Seamless loops break
- Patent licensing until 2017
- Vorbis did the job for free

### Still in use

PlayStation emulators still decode the console's ADPCM samples. A fun read on its sound chip:

[jsgroth.dev/blog/posts/ps1-spu-part-1](https://jsgroth.dev/blog/posts/ps1-spu-part-1)



03

# The modern toolbox

Opus, Vorbis, AAC, XMA, ATRAC9, and how to choose

# Codecs in use today

| Codec      | Good for                              | Watch out for                     | Wwise / Unreal              |
|------------|---------------------------------------|-----------------------------------|-----------------------------|
| ADPCM      | Short, frequent SFX                   | Fixed ~4:1; hiss on bright sounds | Both                        |
| Vorbis     | Music, ambience                       | Weaker than Opus at low rates     | Wwise                       |
| Opus       | VO, music, low bitrates               | More decode CPU than ADPCM        | Both                        |
| Bink / RAD | Many voices at once; Unreal's default | Proprietary to Epic               | Unreal                      |
| AAC        | Apple targets                         | Licensing; loop padding           | Wwise (Apple)               |
| XMA        | Xbox, hardware decode                 | Xbox only                         | Wwise; UE Platform Specific |
| ATRAC9     | PlayStation, cheap decode             | PlayStation only                  | Wwise; UE Platform Specific |

Support varies by version and platform. Check your build.

Remember Titanfall's 35 GB? Respawn's later Apex Legends plays 80 to 100 voices at once, all in Bink Audio.

# Vorbis and Opus

The incumbent

## Vorbis

- Royalty-free; 1.0 released in 2002
- Long-time default in Wwise and many engines
- Mature encoders, predictable decode
- Falls behind at low bitrates

The challenger

## Opus

- IETF standard, RFC 6716 (2012)
- SILK for speech, CELT for music
- 6 to 510 kbps; frames from 2.5 to 60 ms
- Best quality per bit at low rates

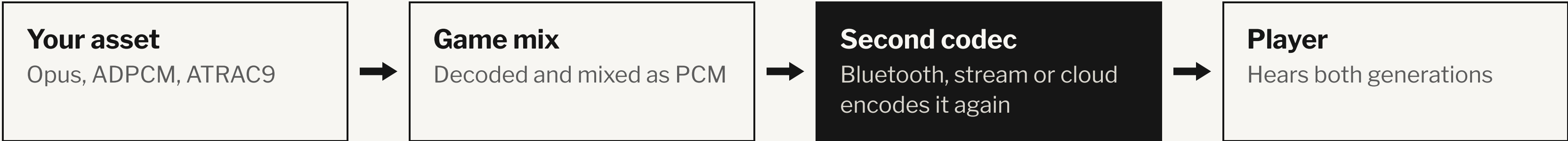
Default new streamed content to Opus. Keep Vorbis where a pipeline already depends on it.

# Platform codecs

| Codec  | Platform       | What you get                               | What you give up        |
|--------|----------------|--------------------------------------------|-------------------------|
| XMA    | Xbox           | Hardware decode; CPU freed for more voices | Xbox only               |
| ATRAC9 | PlayStation    | Low latency and low decode cost            | PlayStation only        |
| AAC    | Apple, Android | Hardware decoders on most phones           | Licensing; loop padding |

Each platform gets its own encode. Budget build time and QA for it.

# Codecs you didn't choose



|                                |                                                                                                                                                        |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Wireless headsets</b>       | Bluetooth re-encodes as SBC, AAC, aptX or LDAC, or LC3 on LE Audio, and adds latency. LC3plus, aimed at gaming headsets, gets that down to about 7 ms. |
| <b>TVs and soundbars</b>       | Bitstream output encodes the mix live as Dolby Digital or DTS.                                                                                         |
| <b>Capture, streams, cloud</b> | The final mix is encoded again, usually as AAC or Opus.                                                                                                |

Lossy generations stack, so leave headroom in the in-game encode.

**Choose a codec per asset  
class, not per project.**

# A decision order

Ask in this order. Stop at the first yes.

|   |                                    |                                         |
|---|------------------------------------|-----------------------------------------|
| 1 | Short, frequent, latency-critical? | <b>ADPCM, or PCM if memory allows</b>   |
| 2 | Dialogue or voice chat?            | <b>Opus, at speech bitrates</b>         |
| 3 | Long and streamed from disk?       | <b>Opus or Vorbis</b>                   |
| 4 | CPU-bound at peak voice count?     | <b>Platform codec: XMA, ATRAC9, AAC</b> |

**On Unreal:** start from Bink Audio, the default, for 1, 3 and 4. Voice chat still needs Opus.

**On Wwise:** Audiokinetic's [A Guide for Choosing the Right Codec](#) has measured CPU and size numbers for every platform.

Then listen on target hardware, at real mix levels, before locking bitrates.

# What to measure

|                             |                                                                     |
|-----------------------------|---------------------------------------------------------------------|
| <b>Decode CPU</b>           | Profile at worst-case voice count, not one sound in isolation.      |
| <b>Memory vs. streaming</b> | Resident assets want cheap decode. Streams want ratio and seeking.  |
| <b>Loops and seeking</b>    | Check that sample-accurate loop points survive encoding.            |
| <b>Bitrate per class</b>    | Set per asset class, then listen on target speakers and headphones. |
| <b>Platform rules</b>       | Check each platform's format requirements.                          |
| <b>Tools and licensing</b>  | Confirm encoder support in the pipeline, and any license terms.     |

04

# The frontier

Sound effects, spatial delivery, voice chat and generated voices

Problem: sound effects in memory

# QOA: the Quite OK Audio format

3.2

bits per sample

5:1

fixed, against 16-bit PCM

≈ 400

lines of C for the reference codec

- An open, MIT-licensed lossy codec by Dominic Szablewski, released in 2023.
- Works in the time domain like ADPCM: it predicts each sample and stores a 3-bit correction.
- By its author's account: better quality than ADPCM, and 3× faster to decode than Ogg Vorbis.
- The format spec fits on one page.

[github.com/phoboslab/qoa](https://github.com/phoboslab/qoa)

## For game audio

- Look for it if you are on Godot (4.3 and later), raylib or a custom engine.
- Try it where you use ADPCM today: short, busy SFX.
- Keep Opus or Vorbis for music and VO. QOA's ratio is fixed.

Problem: delivering a finished spatial mix

# Spatial delivery: APAC and IAMF

From Apple

## APAC

- Apple Positional Audio Codec, built into Apple's operating systems.
- Carries channel layouts and Ambisonics. Required for Apple Immersive Video.
- Apple platforms only, encoded with Apple's tools.

Open and royalty-free

## IAMF

- A container from the Alliance for Open Media. It wraps Opus, AAC, FLAC or PCM.
- Carries channel layouts and Ambisonics, with mixing and loudness instructions.
- Announced for YouTube and Samsung's 2025 TVs, as Eclipsa Audio.

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| What they are for | A finished mix: cinematics, trailers, immersive video.                                |
| What they are not | A runtime format. Your engine still renders the interactive mix.                      |
| What to do        | Find out where your linear content will play, and deliver what that platform decodes. |

Problem: voice chat

# Opus 1.6: neural help for voice chat

9 kbps

fullband speech, matching Opus 1.4 at 18

1 second

of backup audio in every packet

≈ 1%

of a laptop CPU core per feature, or less

|                                |                                                       |                   |
|--------------------------------|-------------------------------------------------------|-------------------|
| Hides lost packets             | Fills the gap with plausible speech                   | Deep PLC          |
| Rides out dropouts             | Carries up to 1 second of backup audio in each packet | DRED              |
| Clearer speech at low bitrates | Repairs the decoded voice and restores its top end    | LACE, NoLACE, BWE |

**What to look for:** all of it is off by default. Ask which of these your engine's or middleware's Opus build turns on.

**What to do:** test voice chat at low bitrates and with packet loss, not only on a clean office network.

[opus-codec.org/demo/opus-1.6](https://opus-codec.org/demo/opus-1.6)

Problem: voice chat

# IVAS: immersive voice from the phone standards

13.2–512

kbps bitrate range

32–38 ms

algorithmic delay

3rd order

Ambisonics, plus objects and 7.1+4

- Immersive Voice and Audio Services: the new 3GPP voice codec, in Release 18.
- Extends EVS, the mono codec phones already use for calls, and stays compatible with it.
- Carries stereo, multichannel, Ambisonics, objects, and a parametric spatial format (MASA).
- Renders inside the codec: speakers, or headphones with head tracking.

[nokia.com: IVAS codec](https://nokia.com/IVAS_codec)

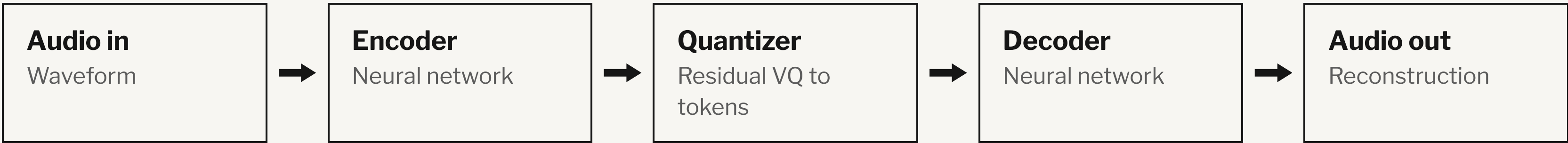
## For game audio

- The natural fit is spatial voice chat: placed, head-tracked voices.
- Aimed at calls, conferencing and XR. Games are not a stated target.
- Like APAC, a delivery codec, not a runtime asset format.

Problem: voice chat

# Neural codecs

Instead of hand-written rules about hearing, a neural network learns from examples how to shrink sound.

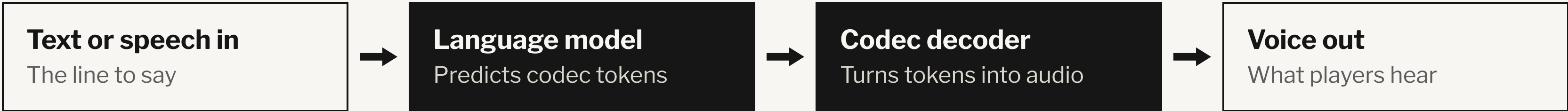


| Codec       | From   | Year    | Notes                                     |
|-------------|--------|---------|-------------------------------------------|
| SoundStream | Google | 2021    | Streamable and causal; about 3 to 18 kbps |
| EnCodec     | Meta   | 2022    | General audio; about 1.5 to 24 kbps       |
| Lyra        | Google | 2021–22 | Speech only; v2 is built on SoundStream   |

**What to listen for:** when they fail, you hear a natural-sounding wrong sound, not hiss or warble.

# AI voices are built on codecs

Many voice generators never produce a waveform. A language model predicts codec tokens, and a neural codec's decoder turns them into sound.



|               |                                                                                        |
|---------------|----------------------------------------------------------------------------------------|
| Bandwidth     | The codec's sample rate caps the result. Ask what rate the voice is generated at.      |
| Latency       | Token rate and lookahead set how fast a line starts. Ask for time to first audio.      |
| Where it runs | On device means one model per voice. In the cloud means a network round trip.          |
| Repeatability | The same line can come back different. That matters for QA, lip sync and localization. |

The decoder invents plausible detail, so its mistakes sound like a different word or voice, not like noise.

# Barriers in games

|                       |                                                                         |
|-----------------------|-------------------------------------------------------------------------|
| <b>Cost per voice</b> | One model inference per stream, and a game mixes dozens of voices.      |
| <b>Latency</b>        | Many models need lookahead. Streaming variants trade quality for speed. |
| <b>Training data</b>  | Mostly speech and music. Sharp SFX transients are unfamiliar.           |
| <b>Determinism</b>    | Float math on GPUs and NPUs can differ across platforms.                |
| <b>Model size</b>     | Weights ship with the game and compete for memory.                      |
| <b>Tooling</b>        | Little middleware support today, and no standard bitstream.             |

# Where neural audio shows up first

|             |                                                                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------------------------------|
| Now         | <b>ML inside classic codecs.</b> Opus 1.5 and 1.6 added neural packet-loss recovery, speech enhancement and bandwidth extension. |
| Next        | <b>Low-bitrate speech.</b> Research codecs reach 20 ms latency at roughly 20× real time on one CPU.                              |
| Speculative | <b>Tokens in place of waveforms.</b> Generative pipelines that store and stream codec tokens, decoded on device NPUs.            |

Expect it in voice before music or SFX.

# Takeaways

---

- 1 Codec choice is a budget decision: CPU, memory, disk and quality at once.
  - 2 Decide per asset class. ADPCM for short SFX, Opus for VO and streams, platform codecs where hardware helps, and Bink Audio as the starting point on Unreal.
  - 3 On the frontier: QOA for sound effects, APAC and IAMF for spatial delivery, and neural tools reaching voice chat first.
-

# Thank you

Questions?

Feel free to get in touch



**LinkedIn**

[linkedin.com/in/cschupman](https://www.linkedin.com/in/cschupman)



**Email**

[collinschupman@gmail.com](mailto:collinschupman@gmail.com)



**GitHub**

[github.com/collinschupman](https://github.com/collinschupman)